2019 IEEE Symposium Series on Computational Intelligence (SSCI)
December 6-9 2019, Xiamen, China

Global Optimal Path Planning for Multi-agent Flocking: A Multi-Objective Optimization Approach with NSGA-III

Adarsh Kesireddy
Department of
Mechanical Engineering
University of Nevada, Reno
Reno, Nevada, United States
Email: akesireddy@nevada.unr.edu

Wanliang Shan
Department of
Mechanical and Aerospace Engineering
Syracuse University
Syracuse, NY, United States
Email: washan@syr.edu

Hao Xu
Department of
Electrical and Biomedical Engineering
University of Nevada, Reno
Reno, Nevada, United States
Email: haoxu@unr.edu

***Abstract*—Flocking is a multi-objective operation performed by multiple agents in uncertain environments. Objectives of flocking include reaching target for each agent, avoiding collision with obstacles and other agents, as well as maintaining certain pattern among all agents. Multi-objective optimization can be performed in priori methods, posteriori methods and scalarizing methods. Pareto front optimization is the best way to optimize multiple objectives simultaneously. To date, flocking has been performed with summation of objective values. In this paper, Pareto front optimization is adopted for the first time for flocking simulation for multi-agents in uncertain environments. For a team of agents, e.g. rovers, in an uncertain environment, Cooperative Co-Evolutionary Algorithm (CCEA) performs well for both exploration and exploitation. CCEAs coupled with Non-Dominated Sorting Genetic Algorithm II (NSGA-II) and NSGA-III are performed to achieve flocking for multi-agents. A new reward structure is introduced for CCEA. In order to check the reward structure, flocking is performed in different environment, open and closed. In addition, the performances of NSGA-II and NSGA-III are compared for various cases of flocking with different numbers of objectives. Towards the end, the effectiveness of the developed methods is demonstrated through numerical simulations.**

***Keywords*—multi-objective optimization; flocking; NSGA-II; NSGA-III; evolutionary algorithm**

I. Introduction

There are major issues with multi-objective optimization solved with scalarizing methods. These include computation complexity and difficulty in obtaining the optimum solution over the entire set of optimum values [1]. Flocking is one of the topics where summation has been used for finding optimum solution. The action of multiple agents/robots/living creatures moving in a pattern with the objective of accomplishing a task is termed as flocking [2]. Scientists from diverse disciplines have been fascinated by the emergence of flocking/swarming/schooling in groups of agents via local interactions [3] [4] [5] [6] [7] [8] [9].

Moving in formation is a difficult task to be performed. In addition, maneuvering around an obstacle adds more complexity to the problem [10]. Even simple interactions between living creatures increase the complexity over larger areas [11]. Some of the examples that best highlight the emergence of such patterns are found in animal motion [8], where animals collectively exhibit some of the most spectacular and fascinating sights in nature. These include flocks of birds turning in unison or migrating in well-ordered formation, schools of fish splitting and reforming as they outmaneuver a predator, herds of large herbivores migrating together, etc.

In 1986, the first computer flocking simulation was conducted by Reynolds with three rules below: [2] [7]

- Flock Centering: attempt to stay close to nearby flock mates.
- Obstacle Avoidance: avoid collisions with nearby flock mates.
- Velocity Matching: attempt to match velocity with nearby flock mates.

There have been various applications of flocking in engineering operations, such as distributed sensing, self-assembly, parallel delivery and military missions with cooperative Unmanned Aerial Vehicles (UAVs) involved [2]. In addition, particle-based flocking is one of the elements of three-dimensional animation technology that has revolutionized the entertainment industry [12].

A single autonomous agent can perform hazardous or remote tasks such as rescue, survey, and retrieval [13]. A single agent can be highly mobile in carrying sophisticated sensors on them. Often a single agent is called upon to perform various task such as, travelling great distances, collecting samples from hazardous environments, and delivering materials with reliability. In comparison, a multi-agent system can perform the above tasks in larger environments with the same amount of time. They can record temporally synchronized, but spatially-separated measurements in a way that a single

978-1-7281-2485-8/19/$31.00 ©2019 IEEE

agent cannot. This system can share resources, or cooperate on complex tasks. However, coordination among a multi-agent team can be difficult. Different coordination mechanisms have been proposed [14], including multi-agent Hall-of-Fame [15], Leniency [16], and Difference Evaluations [17]. Many of these mechanisms work relying on consistent and constant communication between agents.

Optimization is a method of maximizing or minimizing an objective function to achieve desired results. Multi-objective optimization is a procedure of achieving desired results with conflicting objective functions while fulfilling certain specified requirements. Obtaining an optimal solution that satisfies various objectives is not only complex but also not possible in certain scenarios [18].

In this paper, Pareto front optimization techniques are used to find the set of optimal solutions in flocking environments. CCEA coupled with NSGA-II [19] and NSGA-III [20] [21] algorithms is used for multi-objective optimization. In addition, the performance of each technique is also evaluated and compared for flocking. Agents use neural networks to perform action in environments and learning over generations in flocking environments.

The paper is divided into multiple sections as follows: Section II details the background on flocking, multi-objective optimization, and cooperative co-evolutionary algorithm; Section III details methodology development; Section IV summarizes the environment used for flocking; Section V provides results from multiple simulation environments; Section VI concludes this study.

II. Background

In this section, background information of the key concepts for this study including Flocking (Section II-A), Multi-Objective Optimization (Section II-B), and Cooperative Co-Evolutionary Algorithm (Section II-C) is reviewed.

A. Flocking

Control scientists have been showing great interest in analysis of alignment phenomena and consensus problem [22]. Murray and Olfati-Saber [23], and Jadbabaie et al. [24] provided alignment results in networks along with switching topologies. In addition, multiple flocking virtual agents have been introduced [25]. Since then, many research studies have been performed using virtual agents incorporating varying velocity leader, time varying delays, nonlinear dynamics, and consensus control for high-order multi-agent systems.

Low-density environments, where interactions are rarer and flocking formation is more difficult, have been studied [18]. Under low-density conditions, agents should prioritize maintaining the flock over moving towards targets. A new proposal to influencing agent behaviors termed as "follow-then-influence" has been introduced [18].

B. Multi-Objective Optimization

Multi-objective optimization in general can be performed in three different ways: scalarizing methods, priori methods, and posterior methods. Linear scalarization is a process of summation of objective values with random weights. This process converts multiple objectives into a single objective. But there are defects for this process and the main defect for this process can be explained in a simple way. Given the choice between a large quantity of good A and small quantity of good B, or a small quantity of good A and a large quantity of B, an individual might be indifferent to which set of goods is preferred, which generates an indifference curve (Pareto front) with these two points. In linear scalarization, Pareto front will not be used, as the multi-objective solution is converted into a single objective.

$$\text{minimize} \rightarrow \sum_{i=1}^{p} w_i F_i(x), \tag{1}$$

$$\text{subject to} \rightarrow x \in X,$$

where i represents the objective index, $F_i(x)$ is the objective value corresponding to i^{th} the objective, and w_i is the weight corresponding to i^{th} the objective.

To date, optimization of flocking agents has been performed with linear scalarization, along with multiple influencing agents. In this research, Pareto fronts are generated and posteori preference methods are applied to solve multi-objective optimization in the flocking environments.

In multi-objective optimization, the best-known Pareto front should be as close as to the true Pareto front. Solutions should be uniformly distributed with respect to each objective, and Pareto fronts should capture the entire spectrum of solutions over each objective. After achieving the Pareto fronts, NSGA-II and NSGA-III are implemented to obtain optimal solutions from the given set of solutions, respectively [20].

C. Cooperative Co-Evolutionary Algorithm

An Evolutionary Algorithm (EA) is a biologically-inspired adaptive search method, which can be used to develop policies for an autonomous agent. In short, a set of $(n + k)$ random policies are developed, and evaluated to determine their fitness. k policies probabilistically with lower fitness values are then removed. And k new policies are developed using small mutations from the n surviving policies and then added back to the pool.

In a multi-agent system, a CCEA is analogous to multiple interdependent EAs being run in parallel; agents are simulated as team and each agent runs a separate EA in parallel. In a standard CCEA, each group of policies that are simulated share the same fitness. This leads to stable, but not optimal solutions [16]. Previous studies have shown that using credit assignment leads to superior system performance [15].

III. Methodology

In this section we review methodologies applied to obtain flocking: Reward Structure (Section III-A), NSGA-II (Section III-B), and NSGA-III (Section III-C).

A. Reward Structure

Within a CCEA, the fitness assignment is crucial for the learning process of agents. A well-functioning CCEA provides inform about the performance of each agent. In addition, this performance can be used to calibrate the entire system's performance. This is the science of credit assignment. Two simple implementations exist: the local reward and the global reward.

Local Reward: In local reward, each agent is evaluated without considering the performance of other agents in the team. When learning using local reward, the best policy achieved by an individual agent is not communicated to other agents in the team. Due to lack of communication between agents, duplicate efforts in the same policy may be possible. This in turn will affect the performance of the team. Because agents readily seek to enhance their personal benefit and disregard the needs of the system, the system performance can even drop below the performance of randomly-acting agents.

Global Reward: In global reward, each agent is evaluated taking into account the performance of other agents in the team. When learning using global reward, the best policy achieved by an individual agent is communicated with other agents in the team. With constant exchange of performance between agents, the system performance can never drop below the performance of individual agents. The global reward has its own shortcomings, because each agent receives precisely the same evaluation causing the best and the worst policies present in a simulation to receive precisely the same evaluation. In this paper, we use the global reward as not individual but the team should perform to achieve flocking.

B. Non-Dominated Sorting Genetic Algorithm II (NSGA-II)

NSGA-II is the mostly used algorithm in the multi-objective optimization. In this algorithm, agents are created at randomly and evaluated depending upon there performance. Evaluation of agents depends up on the requirement and the reward structure used. Once evaluation of agents is done, Pareto fronts are created using non dominated sorting. Depending up on the number of agents for CCEA, crowding distance is calculated. Algorithm 2, shows later step of selection and re-population.

Creating Pareto front: Pareto front creation is critical in the performance of NSGA-II. Non-dominated sorting is applied on all agents. Each individual agent is compared with other agents in the system and classified as per performance of the agents. During the sorting process, all agents are stored in corresponding fronts using Algorithm 1. An agent is said to be non-dominated by another agent if:

$$f_i(x^n) \leq f_j(x^n), \tag{2}$$

where f_i is the current agent, f_j is another agent and n is the number of objectives.

Algorithm 1: Sorting of N population into Pareto front

```
for n in N do
    S_p = ∅; n_p = 0
    for m in N do
        if n < m then
            S_p = S_p ∪ m
        end
        m<n n_p = n_p + 1
    end
    if n_p = 0 then
        n_rank = 1
        F_1 = F_1 ∪ n
    end
end
i=1
while F_i ≠ 0 do
    Q = ∅
    for n in F_i do
        for q in S_p do
            n_q = n_q − 1
            if n_q = 0 then
                q_rank = i + 1
                Q = Q ∪ n
            end
        end
    end
    i=i+1
    F_i = Q
end
```

Crowding Distance: In CCEA, half of the initial population is reduced and processed for the next generation. During this selection process crowding distance plays a key role. The crowding distance is calculated by creating a cuboid with other agents in the same front. The crowding distance is calculated as shown below:

$$I(d_k) = I(d_k) + \frac{I(k+1).m - I(k-1).m}{f_m^{max} - f_m^{min}}, \tag{3}$$

where $I(d_k)$ is the crowding distance, $k+1$ is the preceding agent, $k-1$ is the post agent, m is the objective function, f_m^{max} is the maximum value of the m^{th} objective for all agents, and f_m^{min} is the minimum value of the m^{th} objective for all agents.

Algorithm 2: NSGA-II in Flocking

```
Result: Final population P_t
Initialize population P_t;
while required_generation do
    Evaluation_of_agents();
    generate_pareto_front();
    while required_population_not_achieved do
        crowding_distance();
    end
    remove_lowperforming_agents();
    mutate();
end
```

C. Non-Dominated Sorting Genetic Algorithm III (NSGA-III)

When the number of objectives reaches more than three, computational cost will increase for NSGA-II. In order to circumvent this issue, a new algorithm was developed known as NSGA-III. The main difference between the two is the process of selecting agents from Pareto fronts. In NSGA-II, the crowding distance is used, whereas in NSGA-III, the process of association to reference line is used.

In NSGA-III, the first population are initialized at random. Once initialization process has been performed, agents are evaluated depending on their performance in the environment. Using Algorithm 1, Pareto fronts are generated. The evaluated fitness values of the agents are normalized. Reference lines are generated using reference points prescribed. Once the reference lines are generated, agents are associated with them, which is a critical step in NSGA-III, as the selection process is performed based on the number of agents associated with the reference lines. Once this is done, normal EA steps, including removing lower performing agents and re-population, are performed as shown in Algorithm 3.

Algorithm 3: NSGA-III in Flocking

```
Result: Final population P_t
Initialize population P_t;
while required_generation do
    Evaluation_of_agents();
    generate_pareto_front();
    while required_population_not_achieved do
        normalize_fitness_values();
        generate_reference_lines();
        find_distance_fitnessValues_referenceLine();
    end
    remove_lowperforming_agents();
    mutate();
end
```

IV. Simulation Environment

In this section, we provide the details about the open environment and the example that was used for thoroughly evaluating the developed algorithms. Firstly, the practical environment is described in Section IV-A. Then, the information about multi-agents used in the simulator is described in Section IV-B. Moreover, the construction of the potential targets and obstacles is given in Section IV-C. Then, the reward structure of multi-objective optimization is generated in Section IV-D and the information of the practical scenario, i.e. Hallway, is given in Section IV-E.

A. Environment

In our study, we conduct the flocking simulations in a two-dimensional finite area with unlimited communication radius as well as unlimited sensors radius. Accordingly the flow of information regrading the environment is not hindered. In addition, unlimited sensing range will allow the agents to acquire their target locations without any blockage from either other agents or obstacles.

Moreover, without loss of generality, all the obstacles in the open environment have been considered to be circles. This will allow the multi-agents to have smooth movements during a period of learning time. In order to calibrate the efficiency of the algorithms and the effectiveness of the reward structure, a more practical scenario, i.e. Hallway, is used. This could increase the complexity of the simulation since it introduces a different type of obstacle and environments into the study. In the open environment, the positions of all the agents and the desired destination are excluded from random initialization of the location of the obstacles. This is because these special locations are critical for ensuring a solution to multi-agent multi-objective optimization exists. The agents are teleported in the simulator, which would reduce computational time as the details of mechanical movements of the agents are not required.

B. Agents Used in Simulator

Each agent used in the simulator is equipped with 12 sensors in total. The detailed setup is shown in Figure 1. Therefore, the area around the agent (360°) can be divided into four quadrants where each quadrant can hold 3 sensors. For instance, the first sensor will be used to detect other agents; the second sensor will be used to detect target locations; and the third sensor will be used to detect obstacles.

If no agent, obstacle or target has been detected in a quadrant, a value of 0 will be assigned to that respective signal. If an agent, obstacle or target has been detected, then the inverse of the distance between the two is assigned to that signal. If multiple agents, obstacles or targets have been detected by the sensors, the inverse of the summation of those distances is assigned to that signal. This can also be represented mathematically as

$$S_i = \sum_j \frac{1}{\delta(i,j)}, \tag{4}$$

where S_i is the signal value for the particular sensor, j corresponds to either agents or obstacles detected, and $\delta(i,i)$ is the distance between an agent and another agent or obstacle.

Each agent has a neural network associated to perform action on the environment. Sensors signal from an agent is provided as input to the neural network, which computes the outputs. If (x, y) is the position of an agent in the environment and δx and δy are the outputs from the neural network, then the new position of the agent in the environment is $(x + \delta x, y + \delta y)$.

C. Targets and Obstacles Location

All the agents have there own designated target location. As shown in Figure 2, location $(2, 0)$ is the initial location of an agent and location $(10, 10)$ is its corresponding target location. Without loss of generality, the obstacles in the simulation are generated randomly.

Target location is considered to have similar dimension of obstacles. This helps in convergence of the learning process of each agent, as well as exploitation period. To check the

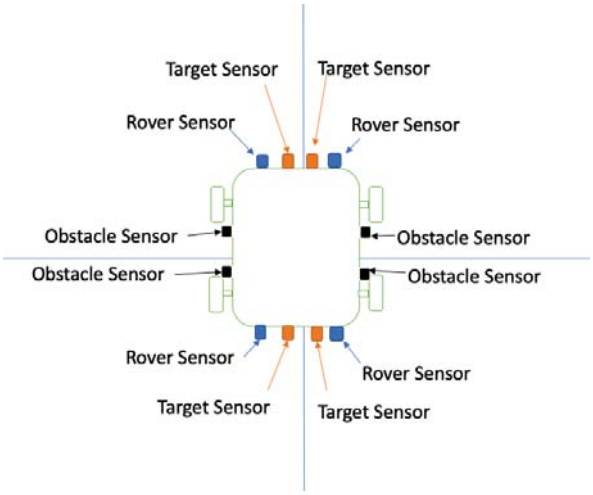

Fig. 1. Senors of agent (three sensors per quadrant)

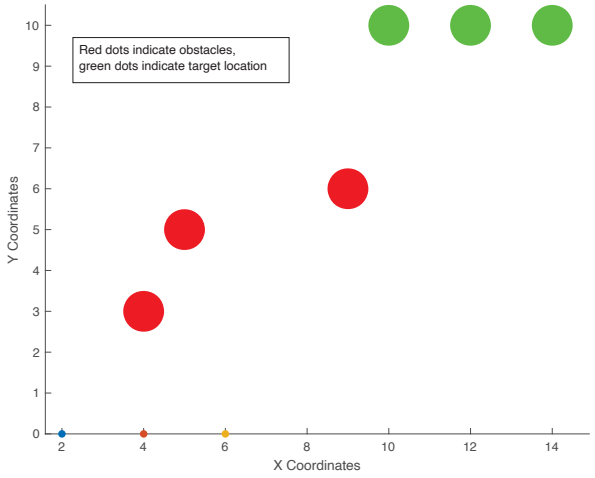

Fig. 2. Open environment of simulation with one set of target locations. Red dots indicate obstacles, green dots indicate target locations.

TABLE I
REWARD STRUCTURE

Objective	Punishment for wrong action
Reaching target	closest distance in the path
Obstacle hitting	1000
Hitting other agent	1000
Following consensus	1000

performance of NSGA-II and NSGA-III, a new set of targets have been introduced in the environment as shown in Figure 3. This will increase the number of objectives from three to four.

D. Reward Structure

In the simulation, each agent will generate a specified number of paths. Each path corresponds to the output of the neural network embedded on the agent. Each path is evaluated to obtain the performance of the agent in the given environment. The reward structure used for simulation is shown in Table I.

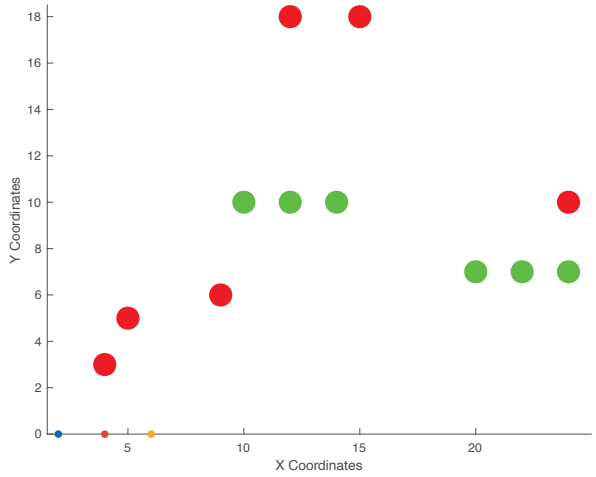

Fig. 3. Open environment of simulation with two sets of target locations. Red dots indicate obstacles, green dots indicate target locations.

As mentioned in Table I, the closest distance between the agent locations and the target locations is considered to be the reward for evaluating whether the agent reaches the target or not. When an agent hits an obstacle or other agents in the path, a punishment valued 1000 will be imposed on the agent. If an agent hits an obstacle and another agent in the same given path, both punishments are summed up. Finally, a punishment of 1000 is imposed on an agent which does not follow consensus per time-step.

E. Hallway

To test generality and robustness of the developed algorithms, a practical scenario, i.e. Hallway, is created. In our environment, since the agents are teleported, the punishment of running into wall is summed per time-step. In the hallway environment, the reward structure is not modified from the open environment. In addition, in the Hallway environment, the agents need to perform a 90° turn to reach the target locations, which will effectively evaluate the capability of the algorithms in complex scenarios such as making turns at sharp corners.

V. RESULTS

In this section, we evaluate the multi-objective optimization algorithms developed for flocking, i.e. NSGA-II (Section V-A), and NSGA-III (Section V-B) coupled with CCEA.

A. NSGA-II

Implementation of NSGA-II coupled with CCEA has been evaluated in the open environment with one target destination. In the first CCEA generation, multiple agents are initialized randomly at the same starting position. Due to randomness in the initialization, the agents explore the environment first. After around 150 CCEA generations, the agents complete exploration and start to exploit the environment. After 10,000 CCEA generations, all the agents can collaborate to maintain flocking. All the three objectives, reaching the destination,

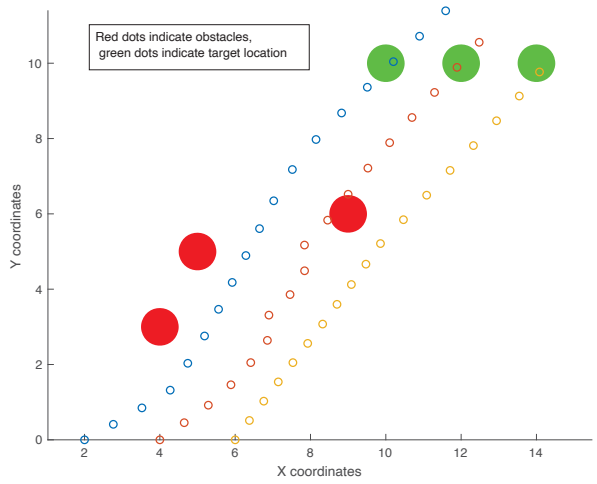

Fig. 4. Flocking movement of agents with one set of target locations at 10,000 generation. Red dots indicate obstacles, green dots indicate target location of agents.

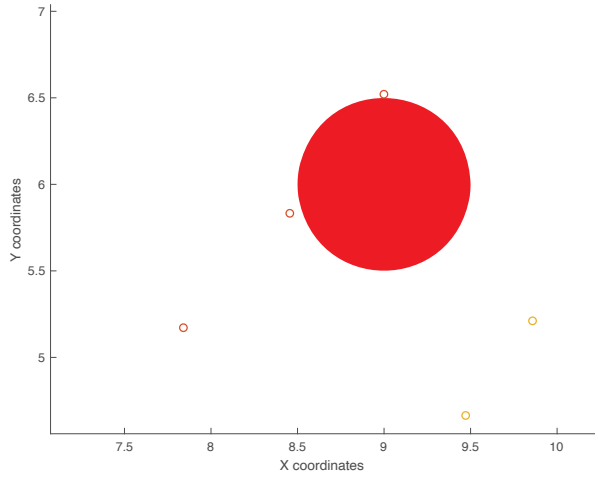

Fig. 5. Zoomed-in view of obstacle avoiding at generation 10,000 from Figure 4.

avoiding collision with other agents or obstacles, and following consensus are accomplished as shown in Figures 4 and 5.

To further investigate the generality and robustness of the developed algorithm, a more complicated and practical scenario has been considered, i.e. multi-agent flocking in the practical Hallway. According to Figures 6 to 8, the developed CCEA can maintain multi-agent multi-objective flocking in the practical Hallway.

Specifically, as shown in Figure 6, multi-agents move in various directions and perform exploration in the first CCEA generation. After the exploration stage, all the agents will enter the exploitation stage and exhibit flocking behaviors. At generation 10,000, multi-agent flocking has been achieved as shown in Figure 7.

In Figure 8, the learning performance of CCEA is demonstrated. The Pareto front of CCEA generation 10,000 is represented with red dots, whereas the blue dots indicates the Pareto front of the first CCEA generation. According

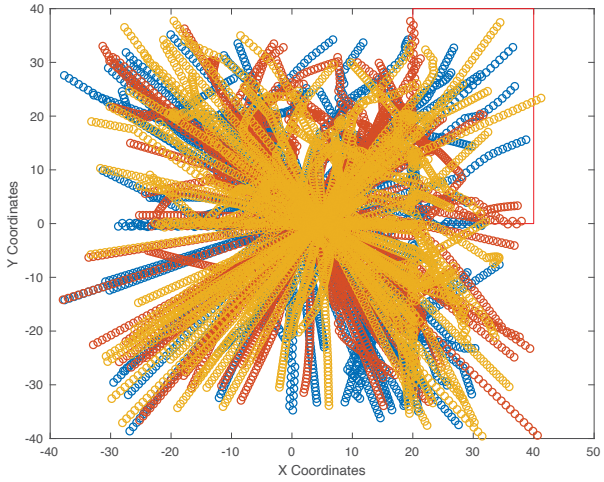

Fig. 6. Paths of flocking agents in Hallway at generation 1.

to the reward structure, the agents that have not performed did receive punishment, which help them in learning and improving performance.

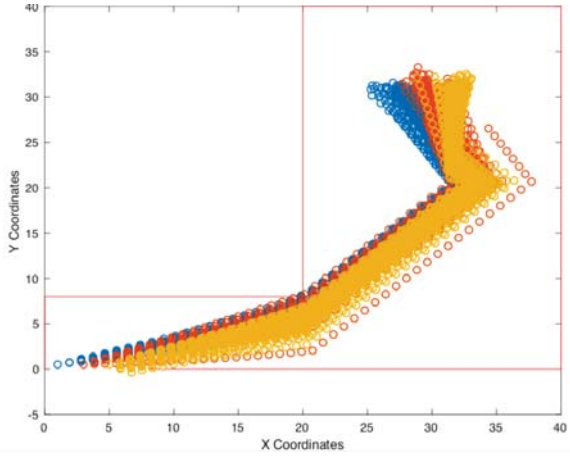

Fig. 7. Paths generating by flocking agents in Hallway, at generation 10000.

Between the open environment and the Hallway, a quicker learning curve was achieved in the Hallway using NSGA-II coupled with CCEA. The main reason for this is that the agents need lower computation on exploration in Hallway scenario, whereas other general open environments need more computation at the exploration stage.

Besides, we studied the effectiveness of CCEA under a different number of agents. Firstly, we increased the number of agents to five without changing the structure of the reward function. As shown in Figure 9, all the agents accomplished all the required objectives for flocking. Similar to before, Pareto curves have been analyzed to evaluate whether the multi-agents have reached all the objectives as expected.

B. NSGA-III

To further investigate the generality and robustness of CCEA and the reward structure, two sets of targets were

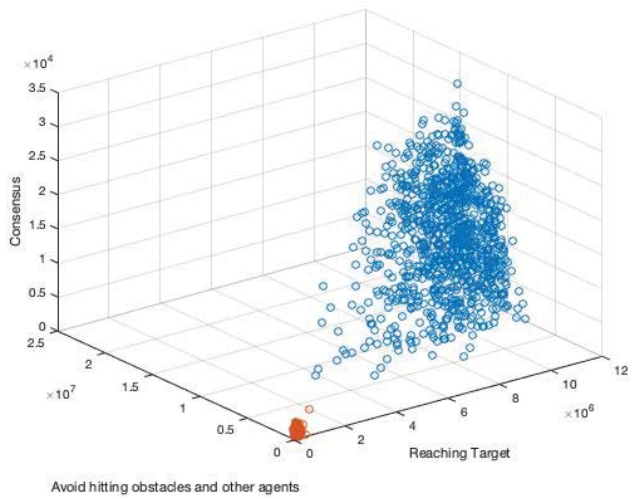

Fig. 8. Pareto front of agents over generation 1 to 10,000 in Hallway.

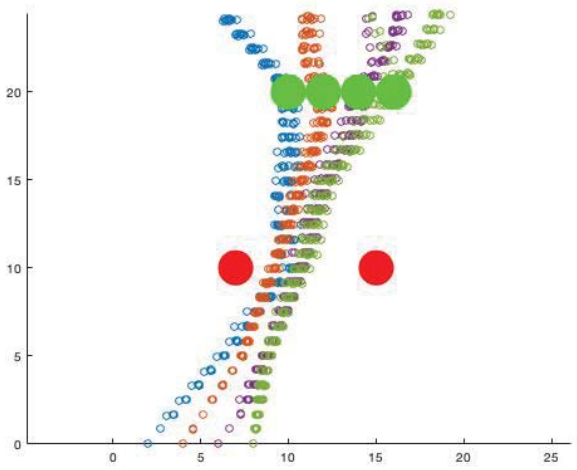

Fig. 9. Paths generated by flocking 5 agents in open environment.

included in the open environment. The additional set of targets will increase the number of objectives for the multi-agent multi-objective optimization from three to four, because all the agents are required to reach the two sets of target locations eventually. In this scenario, both NSGA-II and NSGA-III techniques are used. Specifically, NSGA-III has been able to achieve all the four objectives within 25,000 CCEA generations, whereas NSGA-II has been able to achieve the same objectives within around 80,000 CCEA generations. Therefore, different multi-objective optimization technologies may have different computational complexity even though both of them can achieve the same optimal solution eventually.

Figure 10 shows multi-agents movement at the first CCEA generation. At CCEA generation 25,000, all the agents have been able to learn and reach targets as shown in Figure 11. To evaluate the performance of the inter-agent collision avoidance

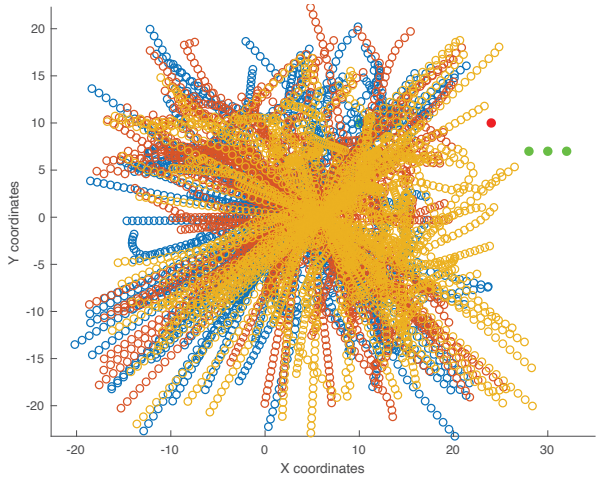

Fig. 10. Flocking movement of agents with two sets of target locations at generation 1.

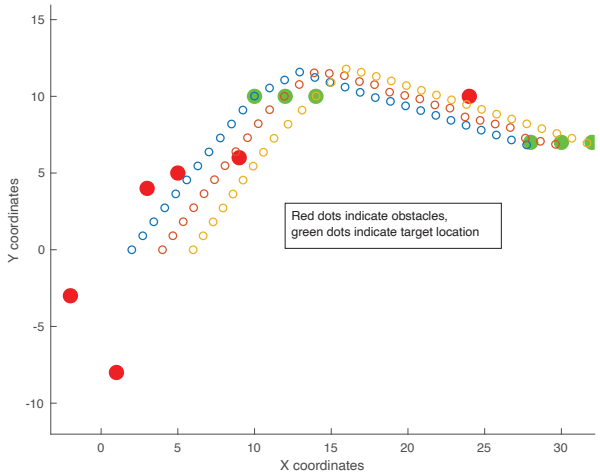

Fig. 11. Flocking movement of agents with two sets of target locations at generation 25,000. Red dots indicate obstacles, green dots indicate target location of agents.

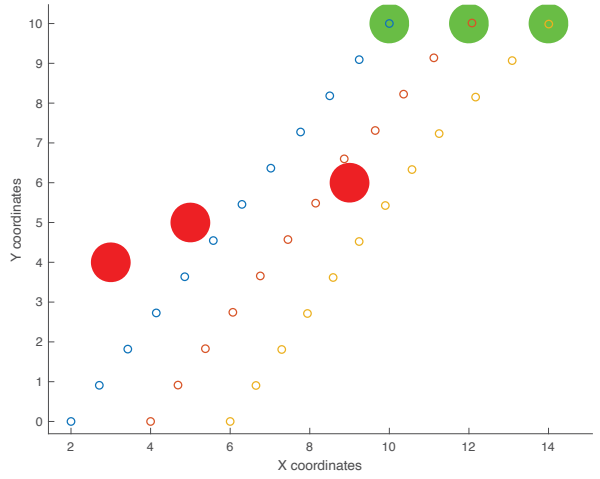

Fig. 12. Zoomed-in view of agents avoiding obstacles in their respective paths.

objective, the Pareto front number has been checked. In addition, Figure 12 indicates that the agents were able to avoid collisions with the obstacles.

VI. Conclusion

In this paper, algorithms combining CCEA with NSGA II and III have been developed for solving a multi-objective multi-agent optimal flocking problem in both open and closed environments. The developed algorithms not only successfully force all the agents to reach multiple desired destinations, but also maintain them in a flocking manner. Furthermore, the developed algorithms can ensure the paths for multi-agents are an optimal solution based on the results from the Pareto front analysis. To better implement CCEA, NSGA-II and NSGA-III have been customized by introducing a new reward structure. Comparison between these two optimization techniques in the open environment, we found that NSGA-III outperformed NSGA-II when the number of objectives increased.

References

[1] I. Y. Kim and O. L. de Weck, "Adaptive weighted-sum method for bi-objective optimization: Pareto front generation," *Structural and multidisciplinary optimization*, vol. 29, no. 2, pp. 149–158, 2005.

[2] R. Olfati-Saber, "Flocking for multi-agent dynamic systems: Algorithms and theory," tech. rep., CALIFORNIA INST OF TECH PASADENA CONTROL AND DYNAMICAL SYSTEMS, 2004.

[3] E. Shaw, "Naturalist at large-fish in schools," *Natural History*, vol. 84, no. 8, p. 40, 1975.

[4] B. Partridge, "The chorus-line hypothesis of maneuver in avian flocks," *Nature*, vol. 309, no. 6, pp. 344–345, 1984.

[5] B. L. Partridge, "The structure and function of fish schools," *Scientific american*, vol. 246, no. 6, pp. 114–123, 1982.

[6] A. Okubo, "Dynamical aspects of animal grouping: swarms, schools, flocks, and herds," *Advances in biophysics*, vol. 22, pp. 1–94, 1986.

[7] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," in *ACM SIGGRAPH computer graphics*, vol. 21, pp. 25–34, ACM, 1987.

[8] T. Vicsek and A. Zafeiris, "Collective motion," *Physics reports*, vol. 517, no. 3-4, pp. 71–140, 2012.

[9] N. Shimoyama, K. Sugawara, T. Mizuguchi, Y. Hayakawa, and M. Sano, "Collective motion in a system of motile elements," *Physical Review Letters*, vol. 76, no. 20, p. 3870, 1996.

[10] A. B. Jones and J. M. Smith, "Article Title," *Journal Title*, vol. 13, pp. 123–456, March 2013.

[11] M. C. Miguel, J. T. Parley, and R. Pastor-Satorras, "Effects of heterogeneous social interactions on flocking dynamics," *Physical review letters*, vol. 120, no. 6, p. 068303, 2018.

[12] H. Liang, J. Sit, J. Chang, and J. J. Zhang, "Computer animation data management: Review of evolution phases and emerging issues," *International journal of information management*, vol. 36, no. 6, pp. 1089–1100, 2016.

[13] Y. Liu and G. Nejat, "Robotic urban search and rescue: A survey from the control perspective," *Journal of Intelligent & Robotic Systems*, vol. 72, no. 2, pp. 147–165, 2013.

[14] C. Guestrin, M. Lagoudakis, and R. Parr, "Coordinated reinforcement learning," in *ICML*, vol. 2, pp. 227–234.

[15] M. Colby and K. Tumer, "Shaping fitness functions for coevolving cooperative multiagent systems," *AAMAS*, vol. 1, pp. 425–432, 2012.

[16] L. Panait, K. Sullivan, and S. Luke, "Lenient learners in cooperative multiagent systems," *AAMAS*, pp. 801–803, May 2006.

[17] A. K. Agogino and K. Tumer, "Analyzing and visualizing multiagent rewards in dynamic and stochastic domains," *Autonomous Agents and Multi-Agent Systems*, vol. 17, no. 2, pp. 320–338, 2008.

[18] K. C. Kapur, "Mathematical methods of optimization for multi-objective transportation systems," *Socio-Economic Planning Sciences*, vol. 4, no. 4, pp. 451–467, 1970.

[19] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.

[20] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[21] H. Jain and K. Deb, "An evolutionary many-objective optimization algorithm using reference-point based nondominated sorting approach, part ii: handling constraints and extending to an adaptive approach," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 602–622, 2014.

[22] H. G. Tanner, A. Jadbabaie, and G. J. Pappas, "Flocking in fixed and switching networks," *IEEE Transactions on Automatic control*, vol. 52, no. 5, pp. 863–868, 2007.

[23] R. O. Saber and R. M. Murray, "Consensus protocols for networks of dynamic agents," 2003.

[24] A. Jadbabaie, J. Lin, and A. S. Morse, "Coordination of groups of mobile autonomous agents using nearest neighbor rules," *Departmental Papers (ESE)*, p. 29, 2003.

[25] H. Su, X. Wang, and Z. Lin, "Flocking of multi-agents with a virtual leader," *IEEE Transactions on Automatic Control*, vol. 54, no. 2, pp. 293–307, 2009.